Test plan for: Co-App

Note that you can refine your testing plan as the project development goes. Keep the change log as follow:

ChangeLog

Version	Change Date	By	Description
1.0.0	March 5th	All members	Initial testing plan Cover Features: 1,2,4
2.0.0	March 27th	All members	Updated testing plan Cover features: 3,5,6 Mutation test

1 Introduction

1.1 Scope

This is our testing scope for Sprint **2**:

1. Feature 1: User authentication
    a. Create account
    b. Log in
    c. Log out
    d. Change password
2. Feature 2: Log Job Applications
    a. Log new application
    b. Delete entry
    c. Update Status
    d. Edit Application Details
3. Feature 4: Company Wiki (“Rate my Co-op”)
    a. Add Company profile
    b. Write Review
    c. Read Reviews

This is our testing scope for Sprint **3**:

4. Feature 3: Application Filtering/Search
    a. Search by Company Fields
    b. Filter by Status
    c. Sort by Date
    d. Change password
5. Feature 5: Interview Calendar
    a. Log interview date
    b. Monthly Calendar View
    c. Quick Navigation
6. Feature 6: AI Resume Helper
    a. Enter a query
    b. Auto-fill job context
    c. Understand the context about me

1.2 Roles and Responsibilities

Each team member takes on one or more of the following roles:

- QA Analyst
- Test Manager
- Backend Developers
- Frontend Developers
- Database Manager
- DevOps

Name	**Net ID**	**GitHub username**	**Role**
Bao Ngo	ngot1	benjaminnNgo	Backend Developer Test Manager DevOps - CI/CD
Aidan McLeod	mcleo16	ACM02	Frontend Developer QA Analyst DevOps - CI/CD
Nikolai Barlaan	barlaann	yomi-adt	Frontend dev QA Analyst
Eric Hodgson	hodgsone	EricHodgson	Backend Developer Test Manager

			Database Manager
Kate Lacerna	lacernak	lacernakate	Frontend Developer QA Analyst
Geet	loombag	gloox	Backend Developer Test Manager DevOps - Deployment

Role Details:

1. A Front-end Developer is responsible for:
    - Create UI components
    - Perform API calls to endpoints according to the API documentation for each feature.
    - Perform code review for front-end PRs
    - Design a sequential diagram up to API calls
2. A back-end developer is responsible for
    - Maintain the database, develop business services and expose endpoint controllers
    - Design API document
    - Perform code review for back-end PRs
    - Add components from the backend to the sequential diagram
3. QA Analyst is responsible for:
    - Test the functionality of the front-end components
    - Perform acceptance tests
4. Test manager is responsible for:
    - Manages and implements tests to achieve 100% code coverage for the backend
    - Implement integration tests
    - Implement load tests.
5. DevOps is responsible for either:
    - Create and maintain CI/CD
    - Deploy application
6. Database Manager
    - Create and host MongoDB database instances for dev and prod environments

2 Test Methodology

2.1 Test level

Test levels define the Types of Testing to be executed on the Application Under Test (AUT). In this course, unit testing, integration testing, acceptance testing, regression testing, and load testing are mandatory.

Feature 1: User authentication - Sprint 1
Unit testing - Automated with regression testing

(JwtAuthFilter) doFilterInternal_whenEverythingSuccess
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/config/JwtAuthFilterTest.java#L68C3-L86C4
Check if the JWT token is parsed successfully

(AuthController) createAccount_whenEverythingSuccess_expect200Response
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/controller/AuthControllerTest.java#L178C3-L195C4
Check the register endpoint with the mock all service and mock MVC - successful case

(UserController) updatePassword_whenIncorrectOldPassword_expect401
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/controller/UserControllerTest.java#L89C3-L104C4
Check the update password endpoint with the mock all service and mock MVC - fail case due to incorrect old password

(UserMode constructor) constructor_whenNoIDProvided_expectAutoGenerateID
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/model/document/UserModelTest.java#L95C3-L105C4
Test UserModel's constructor when no ID is provided and expect the ID to be auto-generated

(UserRepository) findUserModelByEmail_whenFindByJohnEmail_expectJohnUserModel
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/repository/UserRepositoryTests.java#L62C3-L73C4
Test UserRepository with a MongoDB instance in the test container. Test to query a user with email.

(AuthService) createNewUser_whenNewEmailIsUsed_expectEmailSentWith
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/service/AuthServiceTest.java#L108C3-L133C1
Test the business logic of registering a new account.

(EmailService) sendEmail_whenSendSuccessfully_expectNoException

https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/service/EmailServiceTest.java#L87C3-L98C4
Test the business logic of sending an email.

(UserService) updateUserPassword_whenOldPasswordMatch_expectNoException
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/service/UserServiceTest.java#L135C3-L145C1
Test the business logic of updating the password.

Integration testing

(Feature 1 + Feature 2)
applicationFlow_whenUserLogsInAndCreatesUpdatesDeletesApplicationAndLogsOut_expectCorrectDataInDB
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/_integration/ApplicationLifecycleCrossFeatureIntegrationTest.java#L88
Tests the full lifecycle of a job application: create, update, and delete, integration testing with the Authentication, Application Management, and Company features together.

(Feature 1 + Feature 4)
reviewFlow_whenUserLogsInAndPostsReviewAndLogsOut_expectCorrectDataInDB
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/_integration/CompanyReviewCrossFeatureIntegrationTests.java#L84
Tests that an authenticated user can post a review for a company, integrating Authentication, Review Management, and Company Profile features, verifying the review is persisted and the company's average rating is updated correctly.

(Feature 1 + Feature 4)
reviewFlow_whenUserLogsInAndCreatesDuplicatesAndUpdatesReviewAndLogsOut_expectCorrectDataInDB
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/_integration/CompanyReviewCrossFeatureIntegrationTests.java#L154C7-L154C100
Tests that an authenticated user cannot submit duplicate reviews but can update an existing one, integrating Authentication, Review Management, and Company Profile features, verifying duplicate rejection, correct persistence, and accurate average rating recalculation.

(Feature 1 + Feature 2 + Feature 4)
wikiFlow_whenUserLogsInAndSearchesPaginatesReviewsAndAppliesAndLogsOut_expectCorrectDataInDB
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/_integration/SearchPaginationCrossFeatureIntegrationTest.java#L95C7-L95C99

Tests that an authenticated user can search and paginate companies, leave a review, and create a job application from the company profile, integrating Authentication, Company Search, Review Management, and Application Management features together, verifying that all data is correctly persisted and consistent across features.

(Feature 1 + Feature 2 + Feature 4)
wikiFlow_whenUserLogsInSearchesAppliesMultipleAndProgressesStatusAndLogsOut_expectCorrectDataInDB
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/_integration/WikiSearchAndApplyCrossFeatureIntegrationTest.java#L96C7-L96C104
Tests that an authenticated user can search for companies, apply to multiple of them, progress an application status, and delete another application, integrating Authentication, Company Search, and Application Management features together, verifying that only the correct application remains in the database with the updated status.

Acceptance testing

1. Given that I'm on the log-in screen and want to make a new account, when I click the "create new account" button, then I am greeted with a new interface to submit an email and password.
2. Given that I'm looking at the interface to submit an email and password, when I submit a valid email that is not already associated with an account and a "valid" password, then an email will be sent to the email address I submitted that contains a verification code and I'm greeted with a new interface prompting me for that code.
    a. "Valid" password criteria: Must not be empty (must contain at least one non-whitespace character). Cannot start with a whitespace character. Cannot end with a whitespace character
3. Given that I'm looking at the interface to submit an email and password, when I submit an invalid email, I will be shown a message saying that the email I submitted was invalid.
4. Given that I'm looking at the interface to submit an email and password, when I submit an email already associated with an account, then I will be shown a message saying that I must choose a different email.
5. Given that I'm looking at the interface to submit an email and password, when I submit an invalid password, I will be shown a message saying that the password I submitted was invalid.

    a. “Valid” password criteria: Must not be empty (must contain at least one non-whitespace character). Cannot start with a whitespace character. Cannot end with a whitespace character
6. Given that I have access to that email address and see the email containing the verification code, when I take that code and input it into the new interface, then I will have successfully made an account and can log in using the email/password combination that I put in.
7. Given that I have access to that email address and don’t see the email containing the verification code, when I want to receive a new code because I didn’t receive it, then I can press the “re-send code” button on the screen to receive a new code.
8. Given that I’m logged out, have an account created, and on the log-in screen, when I type in my email and password correctly, then I will be logged in, be able to access my data, and perform actions on the application.
9. Given that I’m logged-out, have an account created, and on the log-in screen, when I type my email/password incorrectly, then I will be prompted to try again, my data will be inaccessible, and I will not be able to any actions on the application
10. Given that I’m logged-in and want to log-out, when I click the “log-out” button then I will lose access to my data and will be unable to perform the app’s functionalities
11. Given that I am logged in and want to change my password, when I click the “edit profile” button at the top of the screen, then I will see a page that contains a “change password” button.
12. Given that I’m on the “edit profile” screen, when I click on the “change password” button, then I will see an interface with fields to input my current password and my new password
13. Given that I see the “change password” interface, when I input my current password correctly with a new, valid password, then the password I use to log into this account changes to the new password that I sent.
    a. “Valid” password criteria: Must not be empty (must contain atleast one non-whitespace character). Cannot start with a whitespace character. Cannot end with a whitespace character
14. Given that I see the “change password” interface, when I input my current password incorrectly, then I will see a message saying that I put in an incorrect password
15. Given that I see the “change password” interface, when I input my current password correctly but input an invalid password as my new password, then I will see a message saying that my new password is invalid
16. Given that I see the “change password” interface, when I input my current password correctly but put in that same password as my new password, then I will see a message saying that my new password can not be my old password
17. Given that I’ve successfully changed my password, when I log-in with my new password then I will be granted access to my account and be logged-in

18. Given that I've successfully changed my password, when I log-in with my old password then I will be denied access and see a message saying "wrong password"

Feature 2: Log Job Application - Sprint 1
Unit testing - Automated with regression testing

(ApplicationModelTest) validate_whenAllFieldsValid_expectNoViolations https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/model/document/ApplicationModelTest.java#L68-L72 Verify that the entity validation logic passes when all required fields are correctly set.

(ApplicationModelTest) validate_whenUserIdNull_expectViolation https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/model/document/ApplicationModelTest.java#L74-L80 Verify that the entity validation logic correctly catches null User ID constraints.

(ApplicationRepositoryTest) findByUserId_whenUserHasTwoApps_expectReturnTwo https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/repository/ApplicationRepositoryTest.java#L72-L80 Test the repository integration with a MongoDB test container to ensure custom query methods work correctly.

(ApplicationServiceTest) createApplication_whenValid_expectSuccess https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/service/ApplicationServiceTest.java#L96-L114 Test the main integration flow for creating an application using the database container.

(ApplicationServiceTest) updateApplication_whenNotOwner_expectUnauthorized https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/service/ApplicationServiceTest.java#L194-L211 Test the business logic security to ensure users cannot update applications they do not own.

(ApplicationServiceTest) createApplication_whenDatabaseFails_expectServiceFailException https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/service/ApplicationServiceTest.java#L320-L345 Test the unit logic using mocks to ensure graceful handling of database runtime exceptions.

(ApplicationControllerTest) createApplication_whenValid_expect201AndApplication https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/controller

/ApplicationControllerTest.java#L105-L145 Check the create endpoint with MockMvc - successful case (Happy Path).

(ApplicationControllerTest) updateApplication_whenNotOwned_expect403
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/controller/ApplicationControllerTest.java#L337-L362 Check the update endpoint to ensure correct HTTP 403 Forbidden status when authorization fails.

(ApplicationControllerTest) getApplications_whenValid_expect200AndApplicationList
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/controller/ApplicationControllerTest.java#L590-L603 Check the get all applications endpoint to verify the JSON response structure and status.

Integration testing

(Feature 1 + Feature 2)
applicationFlow_whenUserLogsInAndCreatesUpdatesDeletesApplicationAndLogsOut_expectCorrectDataInDB
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/_integration/ApplicationLifecycleCrossFeatureIntegrationTest.java#L88
Tests the full lifecycle of a job application: create, update, and delete, integration testing with the Authentication, Application Management, and Company features together.

(Feature 1 + Feature 2 + Feature 4)
wikiFlow_whenUserLogsInAndSearchesPaginatesReviewsAndAppliesAndLogsOut_expectCorrectDataInDB
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/_integration/SearchPaginationCrossFeatureIntegrationTest.java#L95C7-L95C99
Tests that an authenticated user can search and paginate companies, leave a review, and create a job application from the company profile, integrating Authentication, Company Search, Review Management, and Application Management features together, verifying that all data is correctly persisted and consistent across features.

(Feature 1 + Feature 2 + Feature 4)
wikiFlow_whenUserLogsInSearchesAppliesMultipleAndProgressesStatusAndLogsOut_expectCorrectDataInDB
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/_integration/WikiSearchAndApplyCrossFeatureIntegrationTest.java#L96C7-L96C104
Tests that an authenticated user can search for companies, apply to multiple of them, progress an application status, and delete another application, integrating Authentication, Company Search,

and Application Management features together, verifying that only the correct application remains in the database with the updated status.

Acceptance testing

1. Given that I'm on a page for showing all the applications I've logged, when I click on a button that says "add new application", then I will see a new interface that will allow me to input a company name, job title, deadline date, and a link
2. Given that I see the interface for submitting the new application, when I input these fields and click a submit button, then a new application will be added to my list with these details
3. Given that I see the interface for submitting the new application, when I input an empty company name or job title ("empty" as in I leave it blank or input a name/job title that is only empty characters) then I will be shown a message saying that I the name/job title is invalid and I will not be allowed to press submit
4. Given that I see the interface for submitting the new application, when I don't select a deadline date then I will see a message saying to select a date and will not be allowed to press the submit button
5. Given that I successfully added a new application, when I return to the page showing the applications I've logged, then I can see the new application within the list.

Feature 4: Company Wiki ("Rate my co-op") - Sprint 1

Unit testing - Automated with regression testing

1. (review controller) createReview_whenValid_expect201AndReview
   https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/controller/ReviewControllerTest.java#L78
   Test creating a new review endpoint with mock services and mock MVC

2. (company controller) createCompany_whenAlreadyExists_expect409
   https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/controller/CompanyControllerTest.java#L332
   Test creating a new company endpoint with mock services and mock MVC

3. (company service)
   createCompany_whenCompanyExistsCaseInsensitive_expectException
   https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/service/CompanyServiceTest.java#L90
   Test creating a new company business logic - when the company already exist

4. (company service) updateAvgRating_whenMultipleReviews_expectCorrectAverage
   https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/service/CompanyServiceTest.java#L223C15-L223C71
   Test updating the rating of the existing review business logic

5. (review service) verifyReviewBelongsToCompany_whenMultipleReviewsForCompany_expectNoException
   https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/service/ReviewServiceTest.java#L494C15-L494C91
   Testing review business logic

Integration testing

(Feature 1 + Feature 4) reviewFlow_whenUserLogsInAndPostsReviewAndLogsOut_expectCorrectDataInDB
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/_integration/CompanyReviewCrossFeatureIntegrationTests.java#L84
Tests that an authenticated user can post a review for a company, integrating Authentication, Review Management, and Company Profile features, verifying the review is persisted and the company's average rating is updated correctly.

(Feature 1 + Feature 4) reviewFlow_whenUserLogsInAndCreatesDuplicatesAndUpdatesReviewAndLogsOut_expectCorrectDataInDB
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/_integration/CompanyReviewCrossFeatureIntegrationTests.java#L154C7-L154C100
Tests that an authenticated user cannot submit duplicate reviews but can update an existing one, integrating Authentication, Review Management, and Company Profile features, verifying duplicate rejection, correct persistence, and accurate average rating recalculation.

(Feature 1 + Feature 2 + Feature 4) wikiFlow_whenUserLogsInAndSearchesPaginatesReviewsAndAppliesAndLogsOut_expectCorrectDataInDB
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/_integration/SearchPaginationCrossFeatureIntegrationTest.java#L95C7-L95C99
Tests that an authenticated user can search and paginate companies, leave a review, and create a job application from the company profile, integrating Authentication, Company Search, Review Management, and Application Management features together, verifying that all data is correctly persisted and consistent across features.

(Feature 1 + Feature 2 + Feature 4)
wikiFlow_whenUserLogsInSearchesAppliesMultipleAndProgressesStatusAndLogsOut_expectCorrectDataInDB
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/_integration/WikiSearchAndApplyCrossFeatureIntegrationTest.java#L96C7-L96C104
Tests that an authenticated user can search for companies, apply to multiple of them, progress an application status, and delete another application, integrating Authentication, Company Search, and Application Management features together, verifying that only the correct application remains in the database with the updated status.

Acceptance testing

1. Given that I'm on the Company Wiki screen and want to create a new company entry, when I click the "Create A New Company Profile" button within the page, then I will be directed to the Create A New Company form to create a new company profile.
2. Given that I have completely filled in the form with valid information and the company has not yet been created, when I select the "Create" option, then I will be redirected to the Company Wiki screen and see the new company I added.
3. Given that I have completely filled in the form with valid information and the company has been created, when I try to create another company of the same name, then I should see a notification saying the company already exists and the company profile could not be created.
4. Given that I have incompletely filled in the form, when I select the "Create" option, then I should remain on the form and the missing fields should be marked in red.
5. Given that I'm on the Company Wiki screen, when I click on a company name, then I should be directed to the company profile.
6. Given that I'm looking at a company profile, when I look at the bottom of the page, then I should see a button to "Add A New Review".
7. Given that I'm looking at the "Add A New Review" button, when I click the button, then I should be directed to the form to add a new review.
8. Given that I have incompletely filled in the form, when I select the "Create" option, then I should remain on the form and the missing fields should be marked in red.
9. Given that I'm on the Company Wiki screen, when I click on a company name, then I should be directed to the company profile.
10. Given that I'm looking at a company profile and there are reviews that exist for that company, when I look at the reviews section, then I should be able to see the reviews about the company.
11. Given that I'm looking at a company profile and there are no reviews that exist for that company, when I look at the reviews section, then I should see a message saying that there's no reviews.
12. Given that I'm looking at the reviews about the company and there are more reviews that are not displayed on the page, when I look at the top-right corner, then I should be able to see a "next page" button.

13. Given that I click the next page button, when I look at the reviews section, then I should be able to see a new set of reviews.
14. Given that I've clicked the next page button and a previous page exists, when I look at the top-right corner, then I should be able to see a "previous page" button.
15. Given that I click the previous page button, when I look at the reviews section, then I should be able to see a previous set of reviews.

Feature 3: Application Filtering/Search - Sprint 3

Unit testing - Automated with regression testing

(Application Controller) getApplications_whenSortParams_expectPassedToService
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/controller/ApplicationControllerTest.java#L776

- Tests passing sort parameters to the service when valid should always pass

(Application Controller) getApplications_whenMultipleStatuses_expectPassedToService
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/controller/ApplicationControllerTest.java#L722

- Tests passing multiple status filters to the service should be able to pass more than one

(Application Service) getFilteredApplications_whenNoFilters_expectAllUserApplications
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/service/ApplicationServiceTest.java#L332

- Test filtering with no filters, which is important for regular fetching without filtering

(Application Service) getFilteredApplications_whenStatusMatches_expectFilteredResults
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/service/ApplicationServiceTest.java#L373

- Test filtering with status filter matching, important it returns filtered results

(GetApplicationsRequest DTO) validateRequest_whenSortByInvalid_expectException
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/dto/request/GetApplicationsRequestTest.java#L163

- Test passing an invalid sort by value to the DTO, which should throw an exception

Integration testing

(Features 1 + 2 + 3)
filterFlow_whenUserCreatesAppliesAndFilters_expectCorrectStatusAndSort
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/_integration/ApplicationFilterCrossFeatureIntegrationTest.java#L78

Tests that an authenticated user can create multiple applications with different statuses, filter them by status, and sort them by date, integrating Authentication, Application Service, and Filter on application feature, verifying that filtering and sorting return correct results and that status updates are accurately reflected in subsequent filter queries.

(Feature 1 + 2 + 3)
wikiFlow_whenUserLogsInSearchesAppliesMultipleAndProgressesStatusAndLogsOut_expectCorrectDataInDB
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/_integration/WikiSearchAndApplyCrossFeatureIntegrationTest.java#L94
Tests that an authenticated user can search for companies, apply to multiple of them, progress an application status, and delete another application, integrating Authentication, Company Search, and Application Service features together, verifying that only the correct application remains in the database with the updated status.

Acceptance testing

- Given that I'm on the "applications" screen, when I type in a search bar, then I will see a list of applications that "match" what I input into the search bar, while applications that don't "match" are hidden
    - "Match" to be decided later. Either "starts with" or some other fuzzy search quality.
    - Currently the plan is to perform the filtering with every keystroke, but if this leads to performance issues then we can have it change to filter on click of a search button
- Given that I have typed something into a search bar, then when I delete the input, then I will see the full list of applications once more
- Given that I'm on the applications screen, when I click on a "filter by status" button, then I will be shown a list of statuses that I can select from
- Given that I see the list of statuses, when I click on one or more of the statuses, then I will only see applications that have statuses that match my selections.
- Given that I see the list of statuses and have previously selected statuses to filter by, when I click on one or more of the statuses that I have previously selected, then I have de-selected those statuses as filters.
- Given that I'm on the applications screen, when I click on an "up arrow" button next to the date applied field, then I will be shown the list of applications in order of date ascending

- Given that I'm on the applications screen, when I click on an "down arrow" button next to the date applied field, then I will be shown the list of applications in order of the date descending
- Given that I'm on the applications screen and have previously selected one of the up or down arrows, when I clicked on the arrow that I selected, then the list will be returned to its "default" ordering
    - Default refers to however the API sends it back the application list

Feature 5: Interview Calendar - Sprint 3
Unit testing - Automated with regression testing

(ApplicationControllerTest) createApplication_whenValid_expect201AndApplication
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/controller/ApplicationControllerTest.java#L115C8-L115C59
Create an application with an interview date by requesting a mock MVC.

(CreateApplicationRequestTest)
validateRequest_whenInterviewDateTooFarInThePast_expectException
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/dto/request/CreateApplicationRequestTest.java#L252C8-L252C72
Test new validation rule for new field: interview date

(ApplicationResponseTest) fromModel_whenValidModel_expectCorrectMapping
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/dto/response/ApplicationResponseTest.java#L75C8-L75C53
Test the application response to check if the new field: interview date is mapped correctly

(ApplicationRepositoryTest) saveNew_expectIdGenerated
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/repository/ApplicationRepositoryTest.java#L67C8-L67C33
Test the application with a new field: Interview Date is saved successfully in the database.

(ApplicationServiceTest)
getInterviewApplications_whenDateRangeProvided_returnFilteredApps
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/service/ApplicationServiceTest.java#L980C8-L980C73

Testing new logic for fetching applications that have interview dates falling under the provided range

Integration testing

(Features 5 + 2 + 1)
interviewFilterFlow_whenUserSchedulesAndFiltersByDate_expectCorrectApplications
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/_integration/InterviewFilterCrossFeatureIntegrationTest.java#L79
Tests that an authenticated user can schedule interviews on applications and filter them by date range, integrating Authentication, Application and Company Service, verifying that the date range filter returns only matching interviews, rejects incomplete parameters, and correctly reflects updated interview dates after rescheduling.

Acceptance testing

- Given that I'm on the Interview Calendar page and I want to log a new interview date, when I click on the option "Add A New Interview", then I should be directed to the form to add a new interview.
- Given that I'm looking at the form to add a new review, when I fill in the form with the company, interview date, and interview time and I click "Submit", then there should be a notification saying that the interview has been successfully created.
- Given that I'm looking at the form to add a new review, when I click "Submit" without completely filling in the form (missing at least one of the following: company, date, and time), then I should remain on the form and the missing fields should be displayed in red.
- Given that I'm on the Interview Calendar page, when I look at the calendar in the page, then the current month should be shown by default.
- Given that I'm looking at the calendar, if I click the next month arrow located on the top right corner of the calendar, then the next month should be shown.
- Given that I'm looking at the calendar, if I click the previous month arrow located on the top right corner of the calendar, then the previous month should be shown.
- Given that I'm looking at the calendar and I have previously logged an interview, when I navigate to the month in which the interview is scheduled, then the interview should be visible in the calendar.
- Given that I'm on the Interview Calendar page and I have upcoming interviews or application deadlines, when I look at the calendar, I should see my upcoming interviews or deadlines listed on their scheduled dates.
- Given that I'm on the Interview Calendar page and I have no upcoming interviews or application deadlines, when I look at the calendar, I should see a message stating that I have no upcoming scheduled interviews or application deadlines.

- Given that I'm on the Interview Calendar page and I have previously logged an interview, when I click on the interview in the calendar, then there should be a popup showing the interview details (company name, date, time).
- Given that I'm looking at the popup displaying the interview details, when I locate and click the X button in the top right corner of the popup, then the popup should close and I should be redirected back to the Interview Calendar page.
- Given that I'm looking at the popup displaying the interview details, when I click outside the popup area, then the popup should close and I should be redirected back to the Interview Calendar page.
- Given that I'm on the Interview Calendar page and I have previously logged multiple interviews on the same day, when I locate the day in the calendar and click on one of the interviews listed for that day, then there should be a popup showing the correct details for the selected interview.

Feature 6: AI Resume Helper - Sprint 3

Unit testing - Automated with regression testing

(UserExperienceRepositoryTest)
findAllByUserId_whenUserHasMultipleExperiences_expectReturnAll
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/repository/UserExperienceRepositoryTest.java#L83C8-L83C70
Check if the experiences are saved properly and can be retrieved successfully from MongoDB

(UserGenAIUsageRepositoryTest)
findUserGenAIUsageModelByUserId_whenUserExists_expectReturnCorrectModel
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/repository/UserGenAIUsageRepositoryTest.java#L50C8-L50C79
Check if the GenAI usage record of a user is saved properly and can be retrieved

(UserExperienceModelTest) model_whenUserIdIsInvalid_shouldFailValidation
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/model/document/UserExperienceModelTest.java#L73C8-L73C54
Check if there is a violation when initializing the User Experience Model with an invalid user ID

(UserGenAIUsageModelTest) model_whenRequestCountIsNegative_shouldFailValidation
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/model/document/UserGenAIUsageModelTest.java#L85C8-L85C61
Check if there is a violation when initializing the User GenAI Usage with negative request count

(UserServiceTest) createNewUserExperience_whenValidArgs_expectReturnSavedModel
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/service/UserServiceTest.java#L292C8-L292C68
Test create experience record logic when all arguments are valid

(GenAIUsageManagementServiceTest) checkAndIncrementUsage_whenUserNotExistInUsageRepoYet_expectIncreaseByOne
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/service/GenAIUsageManagementServiceTest.java#L79C8-L79C81
If there is no GenAI usage record (the user uses the GenAI service for the first time), a fresh GenAI usage record needs to be created and increased by one

(GenAIResumeAdvisorServiceTest) getAdvice_whenUserHasExperience_expectExperienceIncludedInPrompt
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/service/GenAIResumeAdvisorServiceTest.java#L325C8-L325C72
Test to make sure the users' experience context is included in the prompt to GenAI.

(GeminiGenAIServiceTest) generateResponse_whenValidPrompt_expectReturnResponse
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/service/genAI/GeminiGenAIServiceTest.java#L49C8-L49C61
Mock Gemini client and test Gemini wrapper to perform the logic correctly and return expected output

(UserControllerTest) createNewUserExperience_whenSuccess_expect200AndExperienceId
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/controller/UserControllerTest.java#L279C8-L279C68
Test perform query with mock MVC to create a new user experience, expect to create successfully and return experience ID.

(GenAIResumeAdvisorControllerTest) resumeAdvisor_whenQuotaExceeded_expect429
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/controller/GenAIResumeAdvisorControllerTest.java#L208C8-L208C49
Test perform query with mock MVC to get resume feedback from GenAI in the case over usage limit, expect return 429

Integration testing

(Features 6 + 2 + 1)

aiAdvisorFlow_whenUserPromptsWithApplicationContext_expectQuotaDecrementAndCorrectDBState
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/_integration/AIAdvisorApplicationContextCrossFeatureIntegrationTest.java#L80
Tests that an authenticated user can create a job application and use it as context for the AI resume advisor, integrating Authentication, Application Service, and AI Advisor features, verifying that the AI returns a response and the user's remaining quota is correctly decremented after the request.

(Features 6 + 2 + 1)
aiAdvisorFlow_whenApplicationContextIsDeleted_expectNotFoundError
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/_integration/AIDeletedApplicationContextCrossFeatureIntegrationTest.java#L88
Tests that an authenticated user receives a 404 error when using a deleted application as an AI advisor context, integrating Authentication, Application Service, and AI Advisor features, verifying that the AI advisor correctly detects the missing application context and rejects the request after deletion.

(Features 6 + 4 + 1)
experienceFlow_whenUserAddsExperienceForNonExistentCompany_expectFailureThenSuccess
https://github.com/Co-App-Team/backend/blob/main/src/test/java/com/backend/coapp/_integration/ExperienceCompanyConstraintCrossFeatureIntegrationTest.java#L67
Tests that an authenticated user receives a 404 error when adding experience with a non-existent company, then successfully adds the experience after creating a valid company, integrating Authentication, User Experience (Feature 6), and Company features, verifying that experience creation is correctly gated on company existence.

Acceptance testing

- Given that I'm on the main page, when I select the option "Improve my resume with AI" located on the navbar, then I should be directed to the conversation UI like ChatGPT.
- Given that I'm on the conversation UI, when I copy and paste sections of my resume into the prompt and ask the AI chatbot to perform the tasks, then I should be able to see the feedback from the chatbot about my resume.
- Given that I'm on the main page, when I select the option "Improve my resume with AI" on the navbar, then I should be directed to the conversation UI like ChatGPT.
- Given that I'm on the conversation UI, when I click on the option "Select a job", then I should see a selection of the jobs that have been listed so that I can ask the AI chatbot for specific feedback for the selected job listing.

- Given that I clicked on the option "Select a job" and I see the listed jobs, when I select one of the jobs, then I will receive specific feedback from the chatbot regarding tailoring my resume to that job.
- Given that I'm on the main page, when I select the option "Improve my resume with AI" on the navbar, then I should be directed to the conversation UI like ChatGPT.
- Given that I'm on the conversation UI, when I perform prompting with GenAI as in feature 1, I should receive feedback from the AI chatbot with the context of my profile.

Test Level	**Scope & Requirement**	**Methodology (How will you do this?)**
Unit Testing	**Feature 1:** 173 tests **Feature 2:** 100 tests **Feature 4:** 317 tests **~200 tests per core feature.** Total: 590 tests. (Sprint 2) **Feature 3:** 53 **Feature 5:** 19 **Feature 6:** 176 **~82 tests per core feature.** Total: 248 tests. (Sprint 3)	We use JUnit for automated unit testing There are three main components in the backend: Database, Service - containing business logic, and Controller - exposing endpoints to clients. For testing the database, we create a testing MongoDB instance of the database in containers (@Testcontainers) For service, we test business logic by testing the MongoDB instance of the database in containers (@Testcontainers). If the testing service interacts with other services, we mock other testing services. For the controller, each test will spin up a mock MVC instance and perform API calls to the mock MVC instance. If the testing controller interacts with other services, we mock other services.

Integration Testing	**We have implemented 5 integration tests in total to cover interactions between the first 3 features we made and plan to do 5 more when we add the next 3 features.** The code for the current 5 can be found here (one of the files contains 2 tests) and here is a brief description of 3 of the current integration tests: The CompanyReviewCrossFeatureIntegrationTests class contains 2 tests that validate review creation, updates, duplicate prevention, company average rating propagation, and authentication flows. The SearchPaginationCrossFeatureIntegrationTest class contains 1 test that covers searching and paginating companies, followed by creating a review and an application. The ApplicationLifecycleCrossFeatureIntegrationTest class contains 1 test that verifies the full application lifecycle, including creation, updating, and deletion, while ensuring company and user data persistence.	We use @SpringBootTest to load the full application context, @AutoConfigureMockMvc to enable realistic API endpoint testing via MockMvc, and @Testcontainers to spin up isolated MongoDB instances per test class. Each test simulates complete end-to-end user flows, such as logging in, searching or viewing companies, creating or updating reviews and applications, deleting resources, and logging out across features like authentication, companies, reviews, and applications. Tests assert HTTP responses, status codes, and JSON paths in the responses; they extract cookies and JWTs to maintain session continuity; and they directly verify persisted data, counts, and integrity in repositories such as companyRepository and reviewRepository. Cross-feature integrity is ensured by asserting that counts in unrelated collections remain unchanged. All tests are executed automatically via GitHub Actions CI on every PR and push to the main and development branches, alongside unit tests, to provide comprehensive coverage.

	AIAdvisorApplicationContextCrossFeatureIntegrationTest contains 1 test that verifies the full GenAI Resume Advisor lifecycle, including authentication, creating an application, submitting a prompt to the bot, and checking for remaining quota. AIDeletedApplicationContextCrossFeatureIntegrationTest is an incremental test upon AIAdvisorApplicationContextCrossFeatureIntegrationTest with an additional step where the user deletes the application, then submits the prompt again, expecting 404. ApplicationFilterCrossFeatureIntegrationTest contains 1 test that verifies the full application lifecycle, including creating multiple applications and then applying different filters. InterviewFilterCrossFeatureIntegrationTest contains 1 test that verifies the full interview log lifecycle, including creating an application with an interview date and fetching the application according to the interview date range. ExperienceCompanyConstraintCrossFeatureIntegrationTest contains 1 test that verifies	

	user experience lifecycle, including adding user experience to an existing company.	
Acceptance Testing	**Feature 1: -** 18 tests **Feature 2: -** 5 tests **Feature 4: -** 15 tests Total: 38 tests. (Sprint 2) **Feature 3: - Nik** 8 tests **Feature 5: - Kate** 13 tests **Feature 6: - Aidan** 7 tests Total: 28 tests. (Sprint 3)	Acceptance testing has been done manually by walking through the acceptance criteria defined above. The walkthroughs, including screenshots, can be found in our frontend repository.
Regression Testing	Unit + Integration tests executed on **every PR and push to the main and development branches**.	We have configured a GitHub Actions CI pipeline to run all tests automatically. We also configured GitHub Actions CI pipeline (Codecov) to generate a code coverage report for every PR to the main and dev branches.

2.1.1 CI/CD Regression Workflow

Briefly describe your automated pipeline.

- Frontend
    - Every commit to both the develop and main branches triggers the CI pipeline running build and lint steps
    - Every pull request with a target branch of main or develop triggers a CI pipeline running build and lint steps. The pipeline must pass before the PR can be merged
- Backend
    - Each commit to either main or dev will trigger
        - A build of the Docker image
        - A Java code formatting check
        - A Gradle build of the backend
        - Running unit and integration tests

        - Codecov test coverage checking
    - Each pull request for the branches main or dev will trigger
        - A build of the Docker image
        - A Java code formatting check
        - A Gradle build of the backend
        - Running unit and integration tests
        - Codecov test coverage checking
    - CD can be triggered manually to build and push the Docker image to DockerHub

2.3 Mutation Testing (Test effectiveness)

Tool:
PITest 1.20.3 with the JUnit 5 plugin (junit5PluginVersion = '1.2.3')

- used in the Gradle build with the info.solidsoft.pitest plugin.

Mutant Generation:
PIT automatically generates mutants by applying the following mutators to all classes under `com.backend.coapp.service.*` except for the classes under `com.backend.coapp.service.genAI.*` since they do not provide as much testing value as the other service tests:

- `NEGATE_CONDITIONALS`: flips == to !=, > to <=, and other similar operations for conditional statements.
- `CONDITIONALS_BOUNDARY`: shifts boundary operators, e.g. >= becomes >
- `MATH`: replaces math operators, e.g. + becomes -
- `VOID_METHOD_CALLS`: removes calls to void methods
- `NULL_RETURNS / EMPTY_RETURNS`: replaces return values with null or empty values
- `TRUE_RETURNS / FALSE_RETURNS / PRIMITIVE_RETURNS`: replaces boolean/primitive return values
- `INCREMENTS`: mutates ++/-- operators
- `INVERT_NEGS`: negates numeric values

Test Strategy:
To achieve the required 100% mutation score, each service test had to be re-written as a pure unit test, since our existing service tests (which test the interesting logic of the application) use SpringBoot containers that are not compatible with PITest due to the time it takes for them to spin up while creating mutants. These tests were written for each of our main service files under `com.backend.coapp.mutation.*`. These tests use Mockito mocks in place of things like real repositories. They eliminate the issue of the spring context startup and give PIT more control

over the execution path for creating and testing mutants. This approach was approved by the professor, since the existing service tests had to essentially be 'rewritten' as unit tests in a separate location for this strategy to work effectively. Some strategies used to kill mutants were:

- Boundary tests were added explicitly to kill `CONDITIONALS_BOUNDARY` survivors, e.g. testing requestCount == monthlyLimit exactly to distinguish >= from >
- ArgumentCaptor was used on the actual saved state of mutated objects rather than relying on return values, killing `VOID_METHOD_CALLS` mutants

Configuration:

```
targetClasses = ['com.backend.coapp.service.*']
excludedClasses = ['com.backend.coapp.service.genAI.*']
targetTests = ['com.backend.coapp.mutation.*',
'com.backend.coapp.service.JwtServiceTest']
```

- Selecting which classes and tests will be run
- `JwtServiceTest` was already in a compatible format, so it was not duplicated
- The files in `com.backend.coapp.service.genAI.*` were excluded because they do not contain code that would meaningfully contribute to mutation tests. The service tests were chosen because they contain real business logic to be tested with mutants.

```
threads = 4
```

- improve runtime by running on 4 JVM forks

```
timestampedReports = false
```

- Always overwrites the same report directory instead of creating a new timestamped folder each run.

```
outputFormats = ['XML', 'HTML']
```

- Report generation formatting for analysis

```
enableDefaultIncrementalAnalysis = true
```

- Store previous results and only re-analyze changed classes/tests, speeding up repeated runs

```
mutationThreshold = 99
```

- Fails the build if fewer than 99% of mutants are killed
- Because there are 2 equivalent mutants, we cannot leave the threshold as 100% since PITest doesn't know they are equivalent (it requires manual verification)

```
coverageThreshold = 0
```

- The mutation tests are not concerned with coverage for our use case, so there is no associated threshold (we set it to 0)

```
jvmArgs
```

- Passed to each forked JVM:
  - `--add-opens, java.base/java.lang=ALL-UNNAMED`: needed by Spring Boot and PIT

- `-XX:+EnableDynamicAgentLoading`: suppresses warnings that we do not want to see

Results:
243 total mutants were created, all of which were killed except for 2 equivalent mutants (explained later). This covers far beyond the required 10 per feature (which would be 60 tests in total). Below are the metrics for our tests, including total metrics, class specific metrics, and mutator specific metrics all given by the PITest report.

Overall:

Metric	Result
Total Mutations	243
Killed Mutants	241
Survived (equivalent mutants)	2

By Class:

Class	Killed	Total	Score
ApplicationService	69	69	100%
AuthService	31	31	100%
CompanyService	12	12	100%
EmailService	12	12	100%
GenAIResumeAdvisorService	23	23	100%
GenAIUsageManagementService	19	20	95%
JwtService	20	21	95%
ReviewService	23	23	100%
UserService	32	32	100%

By Mutator:

Mutator	Killed	Total	Score
NEGATE_CONDITIONALS	129	129	100%

VOID_METHOD_CALLS	50	51	98%
EMPTY_OBJECT_RETURN_VALS	19	19	100%
NULL_RETURN_VALS	17	17	100%
MATH	11	11	100%
CONDITIONS_BOUNDARY	8	9	89%
PRIMITIVE_RETURNS	4	4	100%
BOOLEAN_TRUE_RETURN_VALS	2	2	100%
BOOLEAN_FALSE_RETURN_VALS	1	1	100%

Surviving Equivalent Mutants:

`GenAIUsageManagementService.getNumberOfRequestLeft()`

- **Mutator:** CONDITIONALS_BOUNDARY
- **Mutation:** numberOfRequestLeft < 0 became numberOfRequestLeft <= 0
- **Description:** When numberOfRequestLeft == 0, the original condition (< 0) is false and the value remains 0. The mutant condition (<= 0) is true but assigns 0 to a variable already holding 0. No input produces a different return value between the original and the mutant.

`JwtService.generateToken(UserDetails)`

- **Mutator:** VOID_METHOD_CALLS
- **Mutation:** Removed call to validateUserDetail(userDetails)
- **Description:** generateToken(UserDetails) calls validateUserDetail before the overload generateToken(Map, UserDetails), which also begins with a call to validateUserDetail. Removing this call has no effect because the invalid details are still caught.

2.4 Load Testing

We want to validate that the backend system can **concurrently handle at least 20 users generating a total of 200 requests per minute (RPM)** across core features (authentication, application CRUD, search/filtering, company reviews, interviews, AI quota, and profile experience). This ensures scalability under realistic peak load without errors or excessive latency.

Tools and Prerequisites

- **Tool**: k6 (open-source load testing tool, installed globally via brew install k6 or equivalent).

- **Environment**:
    - Backend: https://coapp-backend-dev.onrender.com (Render paid tier).
    - Test Company ID: 69a4ddcab0a73ab3e5bd5a8b.
    - 20 real test users: testuser1@test.com to testuser20@test.com (password: 123qwe).
- **Script**: src/test/java/com/backend/coapp/_performance/loadtest.js (k6 JavaScript script simulating full user journeys).

Test Scenario

- **Virtual Users (VUs)**: Up to 20 concurrent VUs, each representing a unique real user
- **User Journey per Iteration** (~15 HTTP requests + 6s sleep for controlled throughput):
    1. Login → Get profile.
    2. Create/delete application (CRUD).
    3. Search/filter applications.
    4. Get companies → Create/delete review.
    5. Get interviews (filtered).
    6. Get AI quota → Create/delete experience.
    7. Logout.

(For feature 6, we don't perform load tests on the API that evolve Gemini API call since there is a Gemini usage limit on the free tier version. Instead, we perform load tests on getting AI quota and creating/deleting experiences. We have confirmed this with the instructor.)

- **Load Stages** (total ~2m duration):
    1. 30s | 20 | Ramp-up (gradual user arrival). |
    2. 1m | 20 | Steady-state (peak load). |
    3. 30s | 0 | Ramp-down (graceful shutdown). |

Execution Steps

1. Navigate: cd src/test/java/com/backend/coapp/_performance.
2. Run: k6 run loadtest.js.

3 Terms/Acronyms

Make a mention of any terms or acronyms used in the project

TERM/ACRONYM	DEFINITION
API	Application Programming Interface
AUT	Application Under Test
dev	Development
prod	Production